从规则组合，走向本题约束下的自主贡献

本轮不把“使用缓存算法、加小模型、动态调整”本身当成原创贡献。贡献需要明确对象、可执行机制、可证命题、失败条件与实验证据。这里提出三条主线，并把决策 CPU 成本作为共同约束。

自主贡献	本轮做到什么	当前判断
提前改变合法动作集合	周期回调提前落盘下一 INSERT 的目标；推导即时写费下界；实现并构造正反例	局部命题成立；裸用存在严重尾部，已加风险预算
有方向的跨算子干扰归因	记录“谁挤掉谁”，由实际落盘后读回更新有方向的成本账本	代码与消融已完成；未证明稳定净收益，不默认启用
对不可见命中的不确定性建模	给出相同回调观察、不同真实访问历史的构造；推导兼容历史损失上界	信息边界已核验；公式完成6400组穷举，尚未接入默认调度

原创性的准确表述

这些是围绕本题接口自行推导、实现与验证的贡献候选，不是对世界范围“首次提出”的证明。提前落盘、部分可观测学习、成本敏感缓存已有研究；本轮的具体区别是回调时点带来的可行动作差异、不可逆 BUILD 增长的风险记账，以及方向性的跨算子落盘来源。

实验并未只保留好消息

初版提前落盘在610例中改善平均收益，但出现约一万倍退化，不能作为稳健默认。后续预算版用原610例作回归，再在冻结后生成32例针对性新测试。报告明确区分这两类证据。

所有Score含公开引擎测得的决策时间。以下均为本机合成输入实验，不是官方成绩、评审创新分或全输入最优证明。

贡献一：提前决策
可以改变可行动作集合

接口差异是机会

题面2.4把“禁止落盘插入目标”限定在 onMemoryOverflow 场景；2.5允许周期回调主动提前落盘；on-disk BUILD 分区的 INSERT 会直接追加写盘。因此，在上一条操作结束的周期回调里，下一 INSERT 的目标尚未成为当前插入目标，可以先将它落盘。

命题：下一条 INSERT 的写 I/O 下界

设当前占用U、容量M，下一条INSERT向驻留BUILD分区X增加d字节，X现有s>0字节，且U+d>M。令D=U+d-M；令m为除X外所有正大小驻留分区的最小大小。

等待溢出：X不可淘汰，任意合法方案必须淘汰非空集合V，故 sum(size(V)) >= max(D,m)。下一INSERT前后的新增写费至少 max(D,m)/W。

提前落盘X：当前写s字节，下一INSERT直接追加d字节，新增写费恰为(s+d)/W。因此当 s+d < max(D,m) 时，即时写费优势至少 G=[max(D,m)-s-d]/W。

证明只用非空集合的最小元素下界与容量约束，不依赖未来操作、真实miss_penalty或访问频率预测。提前落盘后下一INSERT不再触发溢出回调，但周期判断本身也花时间。

实现的三道门

检查	原因
确实存在下一 INSERT、目标正大小驻留、预测将溢出	不依赖空分区SPILL；空分区落盘会被引擎忽略
严格版要求其他正大小驻留分区均处于PROBE	聚焦于保护不可增长的跨算子PROBE数据；不是一般BUILD小分区淘汰
G超过实时扫描开销的平滑估计及风险预算允许	避免为了微小I/O收益支付过多判断时间，防止不可逆增长风险失控

边界：比较的是同一个当前状态下、通过下一条INSERT的写I/O。两种方案的末态不同，未知后续读回可反转收益；它不是全局Score优势证明。

官方题面：2.4动作约束、2.5周期回调与next_op

为什么既需要正例，也需要相同前缀的反例

一个具体例子

M=64MiB；算子A处于PROBE，其热分区63MiB驻留；算子B处于BUILD，其分区X已有1MiB。下一条INSERT向X增加1MiB，随后继续探测A。W=R时：

执行方式	写入	随后读取	说明
等待溢出，再恢复A	先写A的63MiB；恢复A前写X的2MiB	读A的63MiB	总I/O为128MiB对应的时间
周期回调先落盘X	写X的1MiB；下一INSERT追加1MiB	A始终驻留，不读回	总I/O为2MiB对应的时间

这里可见的信息只有下一条INSERT。选择保留A的依据是A当前处于PROBE；并没有偷看“随后继续探测A”的真实后续。后续描述只用于构造验证输入。

相同决策前缀，可以有相反结果

把后续改为：X继续长到64MiB，转入PROBE，然后只探测X。提前落盘迫使后续BUILD持续写盘，首次探测还需整分区读回。若W=10000、R=1，读64MiB远比写小分区昂贵。初版610例里最坏VAL/BASE约为10059.64倍。

反例说明：不能把即时写费证明升级为长期性能保证；也不能仅靠“下一个操作可见”判断BUILD最终大小。正反分支在做选择时的信息完全相同。

本轮取舍

保留机制与局部命题作为贡献，但不把裸版本设为稳健默认。先估算最坏的未知增长和首次读回成本，只有已有运行成本可以支撑这笔暴露时才允许提前落盘。

初版源文件保留为OriginalScheduler.cpp，供复现反例；预算修正版另存为BudgetedScheduler.cpp，未覆盖旧算法或旧实验。

贡献一的修正：给不可逆选择分配风险预算

为何BUILD需要特殊处理

BUILD分区一旦落盘，在SWITCH前不能LOAD。后续每次INSERT都将追加写盘。这个成本不能像PROBE阶段的错误淘汰那样立刻靠换入修复。

可计算的暴露上界

当前X大小s，已知下一INSERT增量d，题面保证最终分区不超过M。取 E=(M-s-d)/W + M/R + 1000，其中负的增长项按0处理。E覆盖：此后最多的BUILD追加字节、首次读回，以及首次读回可能发生的最大miss罚时。

定义已观测成本下界 L=total_io_bytes/max(W,R)+miss_count。每个读写字节耗时至少1/max(W,R)，真实miss罚时至少1，所以L不超过本次运行已经产生的I/O与miss成本。

算法永久保留已批准暴露之和C；只有 C+E <= 0.05L 时才批准下一次目标提前落盘，批准后C增加E。L单调不减。无需读取隐藏的真实罚时，也无需知道最终大小。

可证明什么

在每次批准时，已承诺的上述未知增长与首次读回暴露总额，不超过当时已观测成本下界的5%。不返还预算使记账保守，避免重复花同一笔额度。

不能证明什么

这不是Score<=1.05×BASE的竞争比保证。后续PROBE淘汰会改变驻留状态和控制器历史；E只覆盖明确列出的不可逆增长及首次读回阶段，不覆盖所有后续连锁换入换出。5%是本地固定预算参数，没有宣称最优。

CPU也参与自适应

仅在预测即将溢出的INSERT前扫描候选；用本次扫描耗时更新指数平滑值。当局部写费优势不超过max(0.25微秒,2×平滑扫描时间)时拒绝。两次额外时钟读取与判断开销全部在公开引擎Score里。该门槛是工程启发式，不是精确的未来CPU预测。

贡献二：追踪“谁挤掉谁”，而不只统计谁更热

问题：总量统计会掩盖干扰方向

算子A申请内存导致B的分区落盘，B很快读回；如果只增加B的miss数，控制器知道“B缺内存”，却不知道是A反复挤占了B。

新记录与更新方式

每次SPILL记录请求方u、受影响方v、分区ID、时间与是否局部不可避免。若PROBE分区在窗口内读回，且它与当时目标可以共存，则给有向账本D[u,v]增加 s/W+s/R+1；账本按Probe时钟惰性衰减。

候选落盘时，只有当前请求方u对候选所属算子v的历史干扰会提高该候选的保留价值。A挤过B不会自动禁止C回收B。这与统一提高B热度或固定公平配额不同。

实现采用有限软惩罚

归一化尺度为M(1/W+1/R)+1；惩罚乘数上限4，不形成硬配额、不阻止合法腾出内存。BUILD回调仍主动排除当前INSERT目标。

证据与限制

独立610例严格回放中，干扰版记录了29,904次跨算子短间隔读回，跨算子SPILL从82,989次变为78,382次。

但Score几何比只从0.541200变为0.540153，单组公式均值从2.2386变为2.2354，累计决策时间反而增加。当前证据不足以把它默认开启。

这里的“归因”指自己实际发出的SPILL来源，不是严格的反事实因果损失。即使没有这次SPILL，未来也可能因其他访问发生淘汰；可共存也只是可避免性的必要筛选，不能证明一定能避免。

贡献三：先承认不知道什么，再构造稳健目标

命中历史不可辨识的构造

M=64MiB；驻留A、B各20MiB；磁盘C为44MiB；B=10000，不触发周期回调。一条输入连续命中A共128次；另一条连续命中B共128次；随后二者都miss C，且next_op都是再次PROBE C。

两条输入的所有回调状态、全局Hit/Miss、分区大小、阶段、驻留状态和next_op都相同；但真实近期性与频次相反。任何只使用合法接口的确定性调度器只能给出相同动作，不能完整还原真实LRU/LFU。

验证：四个候选均产生相同观察摘要5580485686556766248，动作摘要也相同；构造中的真实命中分区确实不同。摘要核对是实验校验，逻辑证明来自上述逐项相同的信息。

提出的稳健经验损失

把历史分成驻留/阶段集合已知且稳定的区间j；区间中有Hj次无法定位的命中，可命中的集合为Rj。已定位访问次数为mi；对候选淘汰集合V，设单位访问损失为ci>=0。

与观察相容的最坏经验损失为：
Lrob(V)=sum(i in V) mi*ci + sum(j) Hj*max(i in V∩Rj) ci。空交集的最大值取0。

证明：每个区间的Hj次隐藏命中可以任意集中到该区间损失最大的被淘汰候选，达到这个上界；各区间独立，所以相加。若真实可行集有额外约束，这仍是放宽后的保守上界。

原型对6,400组小规模分配逐一穷举，公式全部吻合。它界定的是过去兼容历史的损失，不是未来请求概率或未来Score。区间建立、next_op样本去重与候选搜索尚未接入提交默认策略。

下一步研究点：只在置信区间明显分离时根据热度换规则；信息不足时保留低成本容量规则。已有部分可观测缓存研究，因此原创主张应聚焦本题可见计数与驻留区间约束的具体形式。

先看完整对照，再决定哪些贡献值得采用

第一批独立610例：源文件先冻结再抽新种子；同一GCC C++14与公开引擎；每候选运行3次，用每例Score中位数。组成：20类负载320例、128个参数端点、6类针对性负载96例、32对相同前缀正反分支、2条不可辨识历史。

版本	几何VAL/BASE	公式均值/10	最差VAL/BASE
旧平衡版	0.5663	2.093	1.655
新基础控制器	0.5412	2.239	1.606
加裸目标提前落盘	0.4690	2.418	10,059.64
加方向干扰账本	0.5402	2.235	1.606

均值不能掩盖退化。正反样例特意占据一部分输入；几何均值降低也不能视为真实业务负载或官方数据集的收益。公式均值按单组10分计算，未转换为比赛总分。

预算修正后的610例回归

版本	几何VAL/BASE	公式均值/10	最差VAL/BASE
同轮基础控制器	0.5412	2.239	2.048
风险预算版	0.5410	2.239	1.667
预算+方向账本	0.5401	2.236	1.810

预算规则是看到前一批反例后修改的，所以这里必须叫回归，不能再把同一610例称为该修正的独立留出集。完整逐例原始结果、最差案例与源文件冻结摘要均保留。

新测试、合法性与决策时间的证据

冻结预算版之后的新32例

每对输入先经过相同的、不可共存大分区交替访问，产生可核对的已有I/O；随后一支保留热PROBE，另一支让BUILD继续增长并探测它。这个专门测试验证预算能否开关与承受相反后续，不代表一般分布。

版本	几何VAL/BASE	公式均值/10	最差VAL/BASE
基础控制器	0.9884	0.113	1.000
预算版	0.9885	0.129	1.015
预算+方向账本	0.9885	0.129	1.015

动作与接口检查

初版4,270次严格回放，加预算版1,284次严格回放：无OOM、无非法BUILD LOAD、无当前INSERT目标落盘、miss动作末尾均LOAD当前目标。预算记账全部满足声明的暴露约束。

改变输入中的隐藏miss_penalty而保持操作序列不变，12组初版检查的可见观察与动作不变。提交算法不读取输入、完整序列、N、B或真实罚时。公开引擎只作允许的调度器替换，未移动计时范围。

代码优化有独立对照

旧版与线性扫描版在610例上的动作序列及读写字节完全相同。公开引擎三次中位数累计决策时间分别为292,248.16与252,494.25微秒。它验证实现优化的价值，但不单独充当主要算法创新。

首次miss之后确实会重新判断

基础控制器在严格回放中共发生22,450次事件/窗口评估与7,570次规则切换。触发因素包括阶段变化、128次Probe、32次Miss、短间隔读回；成功规则有保持与回切门槛。每例全局重新训练并不存在。

当前建议与可以复现的交付

当前选择

若重视已见退化控制，可先采用新基础控制器作为比较起点；风险预算版保留为具有本题特定机制的候选。方向账本暂作为消融模块；不把微小均值变化当成已证实收益。裸目标提前落盘只用于研究与复现，不建议默认启用。

文件	用途
BudgetedScheduler.cpp	风险预算候选，默认flags=4959；仅提交这一个文件可提供MySpillScheduler，无main、C++14、标准库
OriginalScheduler.cpp	初版贡献原型及可复现反例；flags=351关闭新贡献，863开启裸提前落盘，1375仅方向账本
experiment.py / budget_experiment.py	编译并接入原样公开引擎；三次重复计算完整Score
uncertainty_envelope.py	稳健历史损失公式与6400组穷举；尚非默认在线模块
fresh_test / funded_test	输入、种子、摘要、正反分支及不可辨识历史构造
*_summary.json / *_runs.jsonl	逐例实跑、动作审计、最差案例、各项开销和冻结摘要

最大规模检查

每例500万操作	预算版VAL/BASE	墙上时间
maxN_growth_4GiB	1.3698	2.31秒
maxN_maxQ_B1_hits	1.0046	3.14秒
maxN_mixed_maxQ	0.7574	4.97秒

大输入每个配置只运行一次，不能用于细微CPU差异的稳定判断；每例墙上时间均须小于60秒。相关原始输入来自先前压力套件，属于规模回归，不是本轮新抽样。

创新贡献的表达

建议把核心写成：分析受限回调造成的动作集合差异，提出有即时写费证据与不可逆增长预算的跨算子提前落盘；分析信息不可辨识性并给出稳健历史损失原型。动态感知是机制的组成部分，不作为空泛创新口号。

与已有工作的区别以及仍需完成的研究

已有工作	借鉴与区别
Saving Private Hash Join / VLDB 2025	借鉴并发算子内存控制；本轮聚焦题面回调时点、增长不可逆与隐藏命中信息，而非统一页布局、压缩或重分区。
Velox内存仲裁	借鉴跨池回收；方向账本追踪本次请求方对特定受影响算子的落盘来源。
LeCaR / CLOCK-Pro	借鉴淘汰后返回记录；本题部分可见、大小可变、重复写费，不能继承原方法的理论保证。
SIEVE / NSDI 2024	借鉴低维护成本；没有声称复现SIEVE或使用它的论文性能数字。
Miss-Triggered Content Cache Replacement / 2025	检索已发现部分可观测、miss触发学习的相关工作，故不能把“只在miss学习”或POMDP本身视为本轮原创。全文访问受429限制，本报告未据其未读细节作算法对比。

Kuiper et al., Saving Private Hash Join, PVLDB 18(8), 2025

Velox Memory Management, official documentation

Vietri et al., ML-based LeCaR, HotStorage 2018

Jiang et al., CLOCK-Pro, USENIX 2005

Zhang et al., SIEVE, NSDI 2024

Miss-Triggered Content Cache Replacement Under Partial Observability, Mathematics 2025, 13(19):3217

研究边界

尚未完成：把驻留区间的不确定性目标接入在线候选搜索；在更大、独立业务轨迹上确认方向账本的净收益；为提前落盘导致的所有后续驻留变化给出竞争分析。现有局部命题、暴露约束与穷举核验不能替代这些工作。

没有官方隐藏数据成绩，没有全球首次证明，也不据本机数据保证专家创新分。